Adding your custom actions is as simple as adding code to a file and dropping it in the right folder. Open the `playground/assistant_actions` folder from the main project folder, and you'll see several files that define the various actions. Open the `file_actions.py` file in VS Code, as shown in listing 6.4.

Listing 6.4 `playground/assistant_actions/file_actions.py`

```
import os

from playground.actions_manager import agent_action

OUTPUT_FOLDER = "assistant_outputs"


@agent_action     #1
def save_file(filename, content):      #2
    """
    Save content to a file.      #3

    :param filename: The name of the file including extension.
    :param content: The content to save in the file.
    """
    file_path = os.path.join(OUTPUT_FOLDER, filename)
    with open(file_path, "w", encoding="utf-8") as file:
        file.write(content)
    print(f"File '{filename}' saved successfully.")      #4
```

#1 This decorator automatically adds the function as an action.
#2 Give your functions clear names that align with the purpose.
#3 The description is what the assistant uses to determine the function, so document it well.
#4 Generally returns a message stating success or failure

You can add any custom action you want by placing the file in the `assistant_actions` folder and decorating it with the `agent_action` decorator. Just make sure to give the function a good name and enter quality documentation for how the function should be used. When the Playground starts up, it loads all the actions in the folder that are decorated correctly and have descriptions/documentation.

It's that simple. You can add several custom actions as needed. In the next section, we'll look at a special custom action that allows the assistant to run code locally.